

```

7  @Test
8  public void testConcatenate() {
9
10     JUnit test = new JUnit();
11
12     String result = test.concatenate("Vikas", "Kumar");
13
14     assertEquals("VikasKumar", result);
15
16 }
17
18}

```

Code-2 is the test case for the `concatenate()` method defined inside the JUnit class in Code-1. The annotation `@Test` at Line 7 is supported by JUnit version 4. To add JUnit version 4, you can click on *Project* directory in the Eclipse IDE and go to *Java Build Path*, before clicking on *Add Library* and then on *JUnit*, where you select *JUnit 4*.

The `assertEquals()` method is a predefined method, and it takes two parameters. The first parameter is called *expected output* and the second is *original output*. If the expected output doesn't match the original output, then the test case fails. To run the test cases, right click the Eclipse code and then click on *Run as JUnit Test*.

Defining the test case for the `multiply()` method of JUnit class (Code-1)

Code-3

```

1 package com;
2
3 import static org.junit.Assert.*;
4 import org.junit.Test;
5
6 public class MultiplyTest {
7
8     @Test
9     public void testMultiply() {
10
11         JUnit test = new JUnit();
12
13         int result = test.multiply(5, 5);
14
15         assertEquals(25, result);
16     }
17
18 }

```

Code-3 is the test case for the `multiply()` method of the JUnit class defined above.

Creating a test suite

A test suite is a combination of multiple test cases. To create a JUnit test suite, you need to click on the following in Eclipse:

File → *Other* → *Java* → *JUnit* → *JUnit Test Suite*

After creating the JUnit test suite, the code will look like what is shown in the Code-4 snippet.

Code-4

```

1 package com;
2
3 import org.junit.runner.RunWith;
4 import org.junit.runners.Suite;
5 import org.junit.runners.Suite.SuiteClasses;
6
7 @RunWith(Suite.class)
8 @SuiteClasses({ ConcatTest.class, MultiplyTest.class })
9 public class AllTests {
10
11 }

```

Understanding the `@Before` annotation

The `@Before` annotation is used to annotate the method that has to be executed before the actual test method gets executed. To understand this, let's look at Code-5.

Code-5

```

1 package com;
2
3 public class Calculator {
4
5     public int add(int x, int y) {
6
7         return x + y;
8     }
9
10    public int sub(int x, int y) {
11
12        return x - y;
13    }
14 }
15 }

```

Now let's create the test case for Code-5. The following code is the JUnit test case for the *Calculator* class defined in this code.

Code-6

```

1 package com;
2
3 import static org.junit.Assert.*;
4 import org.junit.Before;
5 import org.junit.Test;
6
7 public class CalculatorTest {
8

```